Práctica 4 · Conceptos clave

Detección, seguimiento y lectura de matrículas con YOLO y OCR — guía de referencia para la práctica

1. Durante el entrenamiento

Las pérdidas (loss)

La **loss** mide cuánto se equivoca la red; entrenar consiste en hacerla bajar. YOLO tiene tres:

Loss	Qué mide
`box_loss`	Lo bien colocadas que están las cajas: posición y tamaño respecto a la caja real.
`cls_loss`	Lo bien que acierta la clase. Con una sola clase, en la práctica es "¿hay matrícula aquí o no?".
`dfl_loss`	Afina los bordes de la caja (Distribution Focal Loss). Es un refinamiento de la `box_loss`.

Cada una aparece en dos versiones:

- **train loss:** el error sobre las imágenes con las que aprende.
- **val loss:** el error sobre imágenes que no usa para aprender.

Sobreajuste (overfitting)

Pasa cuando la red **memoriza** las fotos de train en vez de aprender qué es una matrícula. Se nota en las gráficas:

```
train loss   baja, baja, baja, baja...       (sigue bajando)
val loss     baja, baja, se estanca, SUBE    <- aquí empieza a sobreajustar
```

`best.pt` se guarda en el mejor punto de val, así que te protege de esto.

IoU (Intersection over Union)

Mide **cuánto coinciden dos cajas**: la que predice la red y la real.

```
IoU = área de la intersección / área de la unión
```

- `IoU = 1`: coinciden exactamente.
- `IoU = 0`: no se tocan.
- Normalmente una detección se considera **correcta** si su IoU es de 0.5 o más.

TP, FP, FN

- **TP (verdadero positivo):** detecta una matrícula que existe.
- **FP (falso positivo):** detecta una matrícula donde no la hay, por ejemplo un cartel o una pegatina.
- **FN (falso negativo):** hay una matrícula y no la detecta.

Precisión y Recall

- **Precisión (P) = TP / (TP + FP).** De lo que detecta, ¿cuánto es correcto? Si es baja, hay muchas falsas alarmas.
- **Recall (R) = TP / (TP + FN).** De las matrículas que hay, ¿cuántas encuentra? Si es bajo, se le escapan muchas.

Las dos compiten entre sí. Si exiges mucha confianza, detecta menos cosas: sube la precisión y baja el recall. Si exiges poca, pasa lo contrario.

mAP (mean Average Precision)

Es **la métrica principal** de un detector. Resume en un número la precisión a todos los niveles de recall: es el área bajo la curva precisión-recall.

- **mAP50:** cuenta como acierto cualquier detección con IoU de 0.5 o más. Es una exigencia "relajada".
- **mAP50-95:** promedia el mAP con IoU desde 0.5 hasta 0.95. Es mucho más exigente, porque pide cajas muy ajustadas, y por eso siempre sale más bajo.

> Para matrículas, un **mAP50 de 0.85 o más** es bueno. Un **mAP50-95 entre 0.5 y 0.6** también es normal.

2. Al analizar el entrenamiento (paso 1.4)

Gráficas que encontrarás en `modelos/runs/matriculas/`:

Archivo	Para qué sirve
`results.png`	Evolución de todas las losses y métricas por época. **Es la principal para el README.**
`PR_curve.png`	Curva precisión-recall. Cuanto más cerca de la esquina superior derecha, mejor.
`F1_curve.png`	El F1 (media de P y R) para cada umbral de confianza. El pico indica el **mejor umbral** para usar en el pipeline.
`confusion_matrix.png`	Con una sola clase, cuántas matrículas detecta frente a cuántas confunde con el fondo.
`val_batch*_pred.jpg`	Ejemplos visuales de predicciones sobre val.

3. Evaluación (paso 1.5)

Train, val y test

- **Train:** para aprender.
- **Val:** para vigilar el entrenamiento y elegir `best.pt`. El modelo no aprende de ella, pero sí influye indirectamente, porque eliges el modelo según su resultado.
- **Test: no se toca** hasta el final. Es la nota objetiva, con imágenes que nunca ha visto. Es la que pones en el README.

4. En el pipeline (parte 2)

Confianza (conf)

Cada detección viene con un número entre 0 y 1: lo seguro que está el modelo. Con `conf=0.5` te quedas solo con las detecciones de 0.5 o más.

NMS (Non-Maximum Suppression)

La red propone muchas cajas solapadas sobre el mismo objeto. El NMS se queda con la de mayor confianza y elimina las que se solapan con ella por encima de un IoU (`iou=0.7` por defecto). Lo hace YOLO

automáticamente.

Formatos de caja

- **YOLO (labels):** `cx cy w h`, centro y tamaño normalizados entre 0 y 1.
- **xyxy (resultados y CSV):** `x1 y1 x2 y2`, esquinas en píxeles. Es lo que usarás con `box.xyxy`.

Detección en dos etapas

1. YOLO general detecta el **coche**.
2. **Recortas** el coche y tu modelo busca la **matrícula** dentro del recorte.
3. Las coordenadas de la matrícula son relativas al recorte, así que tienes que **sumarles la esquina** del coche (`mx1 = x1_coche + mx1_recorte`) para dibujarla en el frame completo.

Ventaja: menos falsos positivos y matrículas más fáciles de encontrar. Inconveniente: si no detecta el coche, tampoco encuentra la matrícula.

Tracking (seguimiento)

La detección sola ve cada frame por separado y no sabe que el coche del frame 10 es el mismo que el del frame 11. El **tracker** asigna un **ID** a cada objeto y lo mantiene entre frames.

- **BoT-SORT** (por defecto) y **ByteTrack**: predicen dónde estará cada objeto en el siguiente frame (filtro de Kalman) y emparejan las detecciones nuevas con los IDs que ya existen.
- `persist=True`: le dice a YOLO que **recuerde los IDs** entre llamadas. Sin esto, los IDs se reinician en cada frame.
- **ID switch:** cuando el tracker se confunde y cambia el ID de un objeto, por ejemplo tras una oclusión. Puede hacer que cuentes un coche dos veces.

Conteo

Para contar se usan **IDs únicos**, no detecciones. Un coche aparece en cientos de frames, pero tiene un solo ID. Por eso se usa un `set()` por clase.

Vídeo

- **FPS:** fotogramas por segundo. El `VideoWriter` tiene que usar los mismos FPS que el original para que el vídeo no vaya acelerado ni a cámara lenta.
- **Códec (fourcc):** formato de compresión. `'mp4v'` para `.mp4` es lo más compatible.

5. En el OCR (parte 3)

OCR clásico frente a VLM

- **OCR clásico** (EasyOCR, Tesseract, PaddleOCR): redes especializadas en detectar y leer texto. Son **rápidos** y ligeros.
- **VLM** (SmolVLM, Florence-2…): modelos de visión y lenguaje. Les pasas la imagen y una pregunta ("¿qué pone en la matrícula?"). Son más flexibles y a veces aciertan más, pero son **mucho más lentos** y pesados.

Preprocesado

Antes del OCR conviene mejorar el recorte: ampliarlo, pasarlo a escala de grises y aumentar el contraste o binarizarlo. Suele mejorar bastante el acierto.

Postprocesado

Consiste en limpiar el texto leído: quitar espacios y símbolos y corregir confusiones típicas (`O`/`0`, `I`/`1`, `B`/`8`). Las matrículas españolas actuales tienen formato `0000XXX`, así que puedes validar con una expresión regular.

Métricas de la comparativa

- **Exact match (tasa de acierto):** porcentaje de matrículas leídas **perfectamente**.
- **CER (Character Error Rate):** porcentaje de **caracteres** erróneos. Es más justo que el exact match: leer `1234ABD` en vez de `1234ABC` es un fallo de 1 carácter sobre 7 (CER de aprox. 14%), no un fallo total.
- **Tiempo de inferencia:** lo que tarda cada modelo por matrícula. Mídelo con `time.perf_counter()` y **sin contar la primera llamada**, que es más lenta porque carga el modelo (*warm-up*).
